



Relatório Técnico do Poupa+

Aplicativo de gestão de finanças pessoais
Implementação, escolhas técnicas e soluções adotadas

Aluno	Vinicios Andrei Mensen
Curso	Análise e Desenvolvimento de Sistemas
Professor	Alysson Borges
Turma	28743 - DESENVOLVIMENTO MOBILE - EAD54-12
Entrega	Implementação do aplicativo definido no Documento de Concepção, Requisitos e Prototipação (Atividade 1)

1. Visão geral

O Poupa+ é um aplicativo de gestão de finanças pessoais construído sobre uma tese simples, definida na fase de concepção: registrar um gasto precisa levar menos de dez segundos. Esta entrega implementa a versão 1 do produto com todos os requisitos essenciais (RF01 a RF09), todos os importantes (RF10 a RF13) e um dos desejáveis (RF14, exportação CSV), além da integração com uma API pública de cotações para cumprir o critério de comunicação com servidores.

2. Processo de desenvolvimento

O desenvolvimento seguiu o cronograma da seção 5.3 do documento de concepção, adaptado ao formato individual:

- **Setup técnico:** criação do projeto Flutter (canal estável), estrutura de pastas em camadas e repositório Git desde o primeiro commit.
- **Núcleo (RF01 a RF08):** modelagem do banco SQLite conforme a seção 5.2 do documento (tabelas `usuario`, `categoria`, `lançamento`, `orcamento`, `meta`, `aporte`), autenticação local, CRUD de lançamentos, painel e extrato.
- **Orçamento e metas (RF09 a RF13):** tetos por categoria com semáforo, alertas visuais, metas com projeção de aporte mensal e recorrências automáticas.
- **Integração com API:** serviço de cotações com cache offline.
- **Testes e ajustes:** 25 testes de unidade e widget cobrindo as regras de negócio RN01 a RN07, a geração de recorrências, a formatação monetária pt-BR e o parse da API; `flutter analyze` sem avisos.

A rastreabilidade entre requisito e tela definida na seção 4.4 do documento de concepção foi mantida: cada tela implementada referencia nos comentários do código os requisitos que atende.

3. Escolhas técnicas e justificativas

3.1 Plataforma: Flutter com Android como alvo primário

A escolha foi feita na Atividade 1 por matriz de decisão ponderada (Flutter somou 126 pontos contra 108 do nativo Android, seção 3.3 do documento). Os fatores decisivos se confirmaram na prática:

- **Ambiente de testes sem custo:** todo o desenvolvimento e a demonstração acontecem no Android Emulator, disponível em Windows, sem exigência de hardware específico.
- **Produtividade:** o hot reload permitiu iterar as telas do app com ciclo de segundos, e o catálogo Material 3 cobriu a maior parte do sistema de design da seção 4.1.
- **Uma base de código:** o projeto compila para Android hoje e mantém o caminho aberto para iOS sem reescrita.

O Android Studio permanece a IDE de referência do projeto (edição, emulador AVD e build), e o build Android usa Gradle (Kotlin DSL) como definido pelo template oficial do Flutter. Ou seja, as ferramentas estudadas na disciplina (Android Studio e Gradle) estão presentes na cadeia de build, com o Dart/Flutter como camada de aplicação, conforme decidido e fundamentado na Atividade 1.

3.2 Arquitetura: camadas com MVVM (RNF10)

```
views (widgets) -> viewmodels (Provider/ChangeNotifier) -> repositórios -> SQLite / API
```

- **core/** : tema com o sistema de design da concepção (paleta #0D6E62 etc.), formatadores pt-BR e validadores das regras RN01 e RN03.
- **models/** : entidades imutáveis com `toMap` / `fromMap`. O estado derivado (percentual de orçamento, projeção de meta) vive em classes de status (`OrçamentoStatus` , `MetaStatus`) puras e testáveis.
- **data/** : um repositório por agregado. A interface exposta não depende do mecanismo de persistência, o que permite trocar o SQLite por um backend remoto (RF15 futuro) sem tocar nas views, como previsto na seção 5.1.
- **viewmodels/** : `SessaoViewModel` (autenticação e onboarding), `FinancasViewModel` (fonte única de verdade dos dados do mês: após cada mutação recarrega resumo, extrato, orçamentos, metas e relatórios de uma vez, evitando estados inconsistentes entre abas), `CategoriaViewModel` e `CotacaoViewModel`.

Uma decisão importante: todos os valores monetários circulam como inteiros em centavos. Ponto flutuante causa erros de arredondamento clássicos em dinheiro (0.1 + 0.2 não dá exatamente 0.3); com centavos inteiros, somas e comparações de orçamento são exatas. A conversão para exibição acontece só na borda (`Formatters.money`).

3.3 Persistência: SQLite com sqflite (RNF03, RNF09)

- Esquema com chaves estrangeiras ativas e `ON DELETE CASCADE` : excluir a conta remove lançamentos, orçamentos, metas e aportes em uma única operação, o que também implementa o direito de eliminação da LGPD (RNF05).
- Constraints no banco reforçam as regras de negócio na camada mais baixa: `CHECK (valor > 0)` (RN01) e `UNIQUE (usuario_id, categoria_id)` no orçamento.
- Gravações críticas são transacionais (RNF09). O melhor exemplo é o aporte em meta: em uma única transação o app grava o aporte, lança a despesa espelho na categoria Poupança (RN07) e marca a meta como concluída se o alvo foi atingido (RN06). Ou tudo acontece, ou nada acontece.
- O consumo do orçamento é sempre calculado sobre os lançamentos do mês corrente, então o reinício mensal (RN05) é automático, sem necessidade de job agendado.

3.4 Integração com API: AwesomeAPI de cotações

Critério do projeto: integração com alguma API para comunicação com servidores.

- **Endpoint:** `GET https://economia.awesomeapi.com.br/json/last/USD-BRL,EUR-BRL,BTC-BRL` (HTTPS/TLS, atendendo o RNF04; API pública brasileira, sem chave).

- **Fluxo:** o `CotacaoService` faz a requisição com timeout de 8 segundos usando o pacote `http`, converte o JSON em objetos `Cotacao` e grava a resposta bruta em `shared_preferences`.
- **Comportamento offline:** se a rede falha, o serviço devolve o cache com a flag `doCache`, e o card do painel exibe um ícone de nuvem cortada com tooltip explicando que o valor é o último salvo. Sem rede e sem cache, o card apenas não aparece. A falha de rede nunca degrada as funções essenciais, em linha com o RNF03.
- O parse é uma função estática pura, coberta por teste de unidade com resposta real de exemplo, incluindo o caso de resposta vazia.

Por que essa API: cotações de moeda são informação financeira útil no contexto do app (o público-alvo sente o câmbio no preço de tudo), a API é estável, gratuita e sem cadastro, o que facilita a demonstração em sala, e o padrão de consumo REST com JSON e cache é exatamente a competência que o critério avalia.

3.5 UX aplicada (critério de design)

- **Fluxo de 3 toques (RNF02/US01):** FAB visível em todas as abas; o formulário abre com o teclado numérico já focado no valor; categoria sugerida pelo último uso e data preenchida com hoje. O campo de valor formata centavos enquanto o usuário digita, no padrão dos apps bancários brasileiros.
- **Primeira dobra responde "posso gastar?" (US02):** saldo do mês em destaque tipográfico máximo, com receitas/despesas e barra agregada de orçamento logo abaixo, sem rolagem.
- **Cor sempre carrega informação (seção 4.1):** verde de receita, vermelho de despesa/estouro, âmbar de alerta em 80%. Valores absolutos acompanham as barras para a informação não depender só da cor (RNF08, daltonismo), e receitas e despesas também se distinguem pelo sinal.
- **Aviso antes do estouro, não depois (US03):** banner âmbar no topo de Orçamentos lista as categorias em atenção; ao salvar o lançamento que cruza 80% ou 100% do teto, um SnackBar avisa na hora.
- **Confirmação para ações destrutivas (RF05):** exclusão de lançamento (gesto de deslizar), meta, categoria e conta pedem confirmação, com a ação destrutiva em vermelho.
- **Acessibilidade (RNF08):** alvos de toque de no mínimo 48 dp (tema com `MaterialTapTargetSize.padded`), contraste da paleta verificado (#0D6E62 sobre branco fica em 5,9:1) e tooltips em todos os botões de ícone.
- **Localização (RNF12):** interface em pt-BR, moeda R\$ com vírgula decimal, datas dd/mm/aaaa e seletor de datas do Material traduzido via `flutter_localizations`.

3.6 Segurança e privacidade (RNF04, RNF05)

- Senhas com SHA-256 e sal aleatório por usuário (pacote `crypto`); nenhuma senha em texto puro no banco.
- Tráfego com o servidor exclusivamente por HTTPS.
- Modo local sem conta: o usuário pode usar tudo sem informar nenhum dado pessoal.
- Exclusão total: a tela de Perfil exclui conta e dados definitivamente, em cascata, atendendo o direito de eliminação da LGPD.

4. Soluções para problemas encontrados

Problema	Solução adotada
Notificações locais (RF10) exigem plugin com configuração nativa extra (desugaring do Gradle), o que fragiliza o build em outras máquinas	O alerta de orçamento virou feedback visual imediato em três pontos (SnackBar ao salvar, banner na tela de Orçamentos, barra agregada no painel), o que cumpre a história US03 (avisar antes do estouro) e é demonstrável em sala; a notificação de sistema ficou como evolução
Recorrências (RF13) precisariam de agendamento em background	Geração na abertura do app: um método idempotente calcula as ocorrências pendentes desde o lançamento-modelo (com ajuste de dia 31 para o último dia de meses curtos) e insere só as que faltam; testes de unidade cobrem os casos de borda
Consistência entre abas após cada lançamento (saldo, orçamento e relatório mudam juntos)	<code>FinancasViewModel</code> como fonte única de verdade: toda mutação dispara um recarregamento completo das projeções do mês
Erros de centavos em somas de dinheiro com <code>double</code>	Valores como inteiros em centavos em todo o domínio; formatação pt-BR apenas na exibição
Categoria com histórico não pode ser excluída (RN04)	O repositório verifica lançamentos vinculados e decide entre <code>DELETE</code> e desativação (<code>ativa = 0</code>), informando o usuário do que aconteceu
API fora do ar durante a apresentação	Cache da última resposta com indicador de offline; o roteiro de demonstração não depende de rede
App travava na splash nativa no emulador x86_64 (primeiro frame nunca renderizava, sem erro no log)	Diagnóstico por logcat e screenshot via adb apontou o renderizador Impeller em OpenGL ES como causa; relançar o app com <code>--ez enable-impeller false</code> confirmou. A correção foi desativar o Impeller no <code>AndroidManifest.xml</code> (meta-data <code>EnableImpeller = false</code>), voltando ao renderizador Skia, que funciona em qualquer AVD

5. O aplicativo em execução

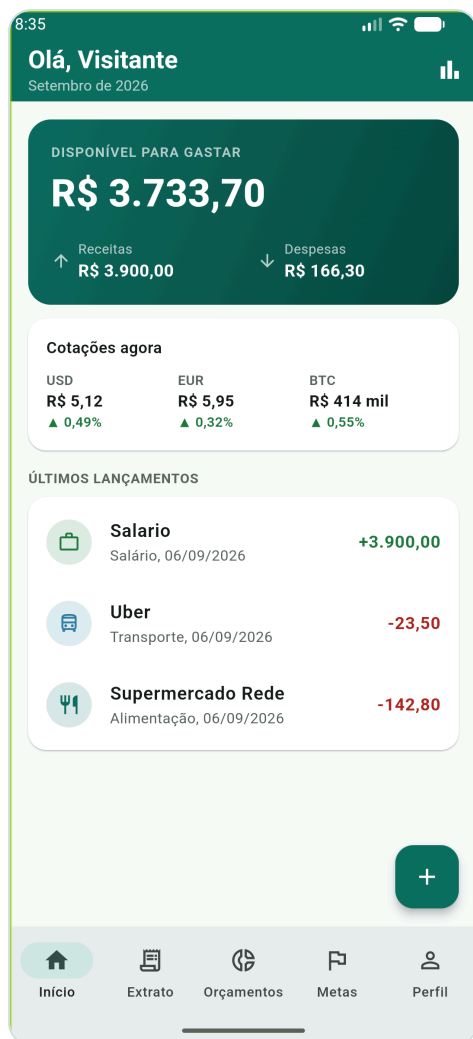
Telas capturadas no Android Emulator (Pixel, API 36), seguindo o roteiro de uso da persona: entrada em modo local, onboarding, lançamentos, orçamento com alerta, meta com aporte e relatórios.



Tela 1. Login, com o modo local sem conta (RF01)



Tela 2. Onboarding com renda e categorias (RF02/RF06)



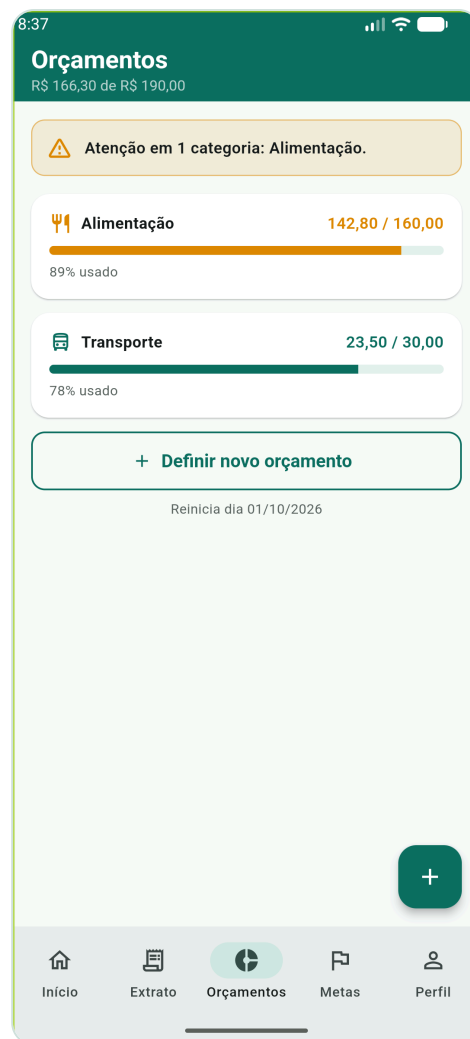
Tela 3. Painel com saldo do mês (RN02), cotações da API e últimos lançamentos (RF07)



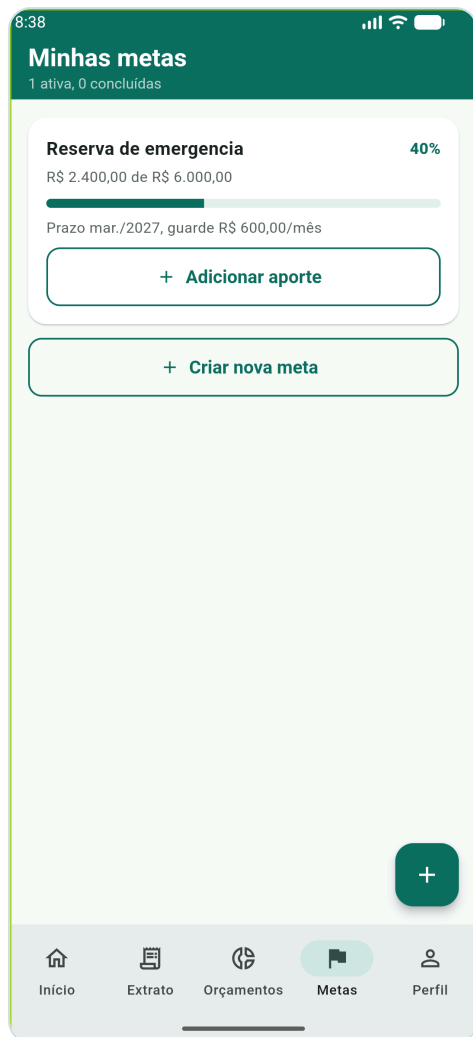
Tela 4. Nova transação em 3 toques, valor em foco (RF03/RF04, RNF02)



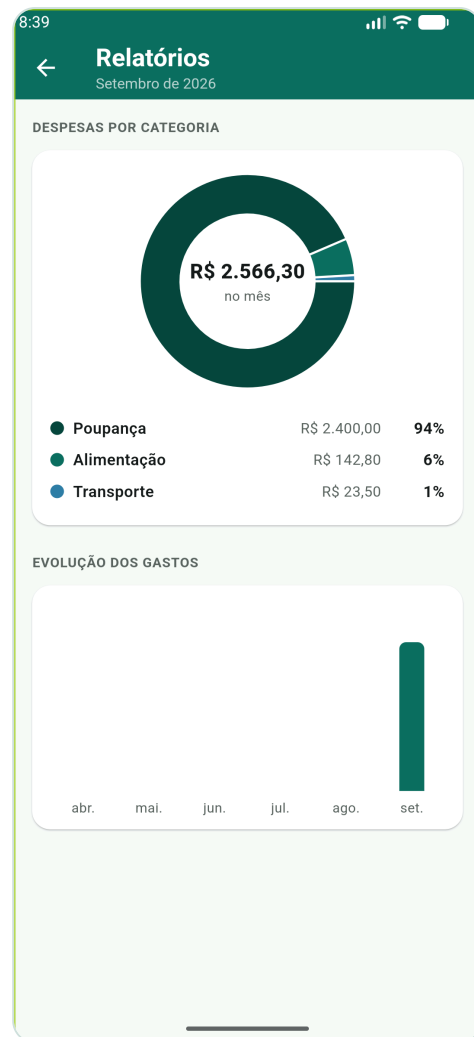
Tela 5. Extrato agrupado por dia, com o aporte como despesa de Poupança (RF08, RN07)



Tela 6. Orçamentos com semáforo: âmbar aos 89% e banner de atenção (RF09/RF10)



Tela 7. Meta com progresso de 40% e aporte mensal recalculado (RF11, RN06)



Tela 8. Relatórios: rosca com total no centro e evolução mensal (RF12)

6. Qualidade e verificação

- `flutter analyze` : zero problemas.
- `flutter test` : 25 testes passando, cobrindo as regras de negócio RN01, RN03, RN05/RF09/RF10 (semáforo), RN06/RF11 (metas e projeção de aporte), RF13 (recorrências, inclusive meses curtos), formatação e parse monetário pt-BR, parse da API de cotações e testes de widget do componente de lançamento (sinais e cores de receita/despesa, RF04).
- `flutter build apk --debug` : build Gradle completo verificado (APK gerado em `build/app/outputs/flutter-apk/`).
- A camada de negócio (classes de status, validadores, formatadores, geração de recorrência e parse da API) é o foco da cobertura, conforme o RNF10.

7. Como executar

- Instalar o Flutter (canal estável) e o Android Studio com um AVD (ex.: Pixel 6, API 34 ou superior).

- `flutter pub get`
- `flutter run` com o emulador aberto (ou `flutter build apk --debug` para gerar o APK).

8. Limitações e evolução futura

- RF15 (sincronização em nuvem) e RF16 (biometria) são desejáveis e ficaram fora desta versão, como previsto na priorização MoSCoW; a arquitetura de repositórios já isola o ponto de troca da persistência.
- Notificações de sistema para os alertas de orçamento (hoje o alerta é visual, dentro do app).
- Recuperação de senha por e-mail exige backend; no modo atual (banco local), a conta é recuperável apenas no aparelho.
- Build iOS a partir da mesma base, via integração contínua em nuvem (sem macOS local).

9. Conclusão

A versão 1 do Poupa+ implementa o escopo essencial e importante definido na concepção, com arquitetura em camadas testável, operação totalmente offline, integração real com API pública e a experiência de registro em 3 toques que motivou o produto. O código está organizado para receber as evoluções mapeadas sem retrabalho estrutural.